

Lecture 11

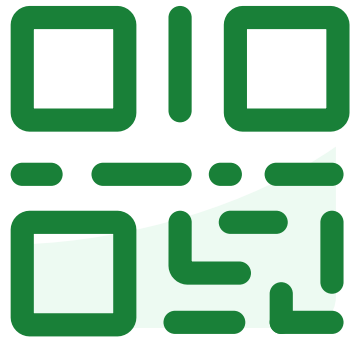
Logistic Regression (2)

Logistic Regression Optimization, Decisions, and Evaluation

EECS 189/289, Fall 2025 @ UC Berkeley

Joseph E. Gonzalez and Narges Norouzi

Do not edit
How to change the
design



**Join at slido.com
#1643395**

① The Slido app must be installed on every computer you're presenting from

slido

Roadmap

- Regularization with Logistic Regression
- Logistic Regression Optimization
- Making Decisions
- Multi-Class Classification
- Model Evaluation



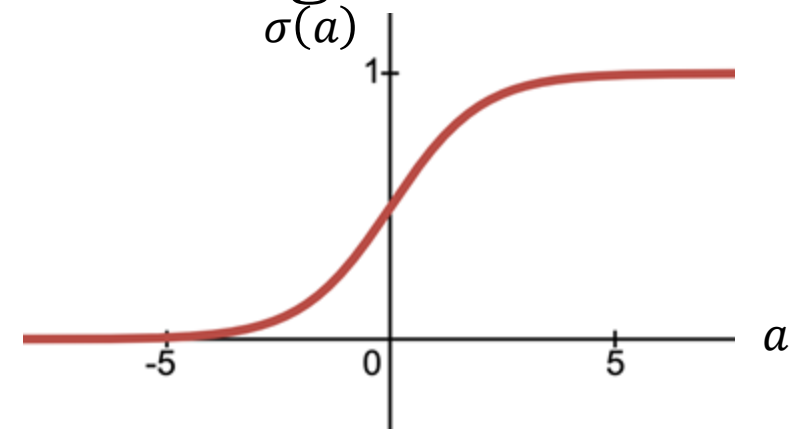
The Sigmoid (σ) Function



1643395

The S-shaped curve is formally known as the **sigmoid function**.

$$\sigma(a) = \frac{1}{1 + e^{-a}}$$



Reflection/
Symmetry

$$1 - \sigma(a) = \frac{e^{-a}}{1 + e^{-a}} = \sigma(-a)$$

Domain
 $-\infty < a < +\infty$

Range
 $0 < \sigma(a) < 1$

Inverse

$$a = \sigma^{-1}(p) = \log\left(\frac{p}{1-p}\right)$$

Logit Function

Derivative

$$\frac{d}{da} \sigma(a) = \sigma(a)(1 - \sigma(a)) = \sigma(a)\sigma(-a)$$

The Logistic Regression Objective (MLE)



1643395

$$p(\textcolor{brown}{t}|\textcolor{violet}{w}, \mathcal{D}) = \prod_{n=1}^N y_n^{\textcolor{brown}{t}_n} (1 - y_n)^{1 - \textcolor{brown}{t}_n}$$

We can define an error function by taking the **negative logarithm** of the **likelihood**, which gives the **cross-entropy error function**.

$$E(w) = -\ln p(\textcolor{brown}{t}|\textcolor{violet}{w}, \mathcal{D}) = - \sum_{n=1}^N [\textcolor{brown}{t}_n \ln y_n + (1 - \textcolor{brown}{t}_n) \ln(1 - y_n)]$$



1643395

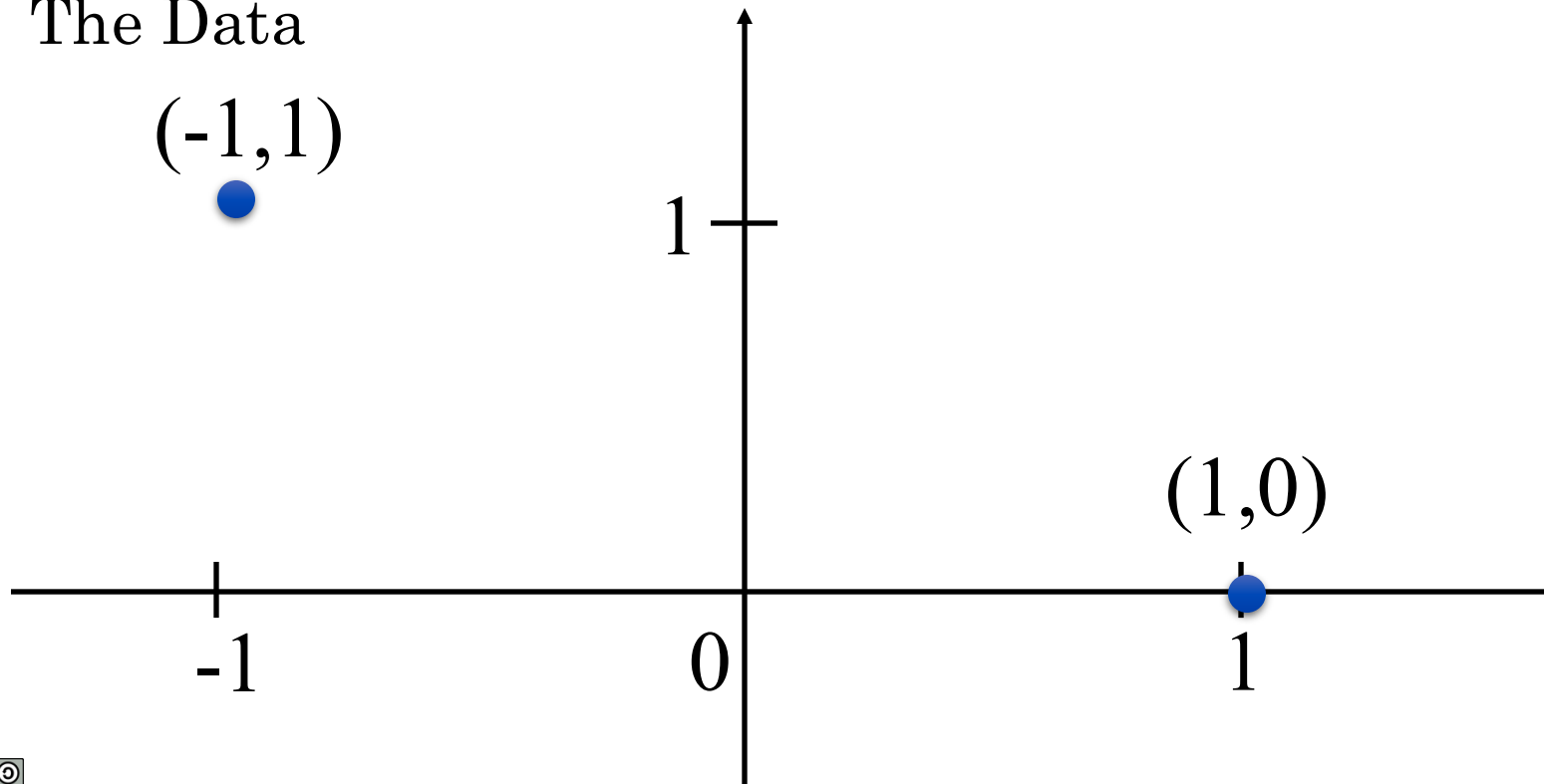
Regularization with Logistic Regression

What Is the Value of w ?

$$w = \underset{w}{\operatorname{argmin}} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

Assume $\phi(\mathbf{x}) = \mathbf{x}$

The Data

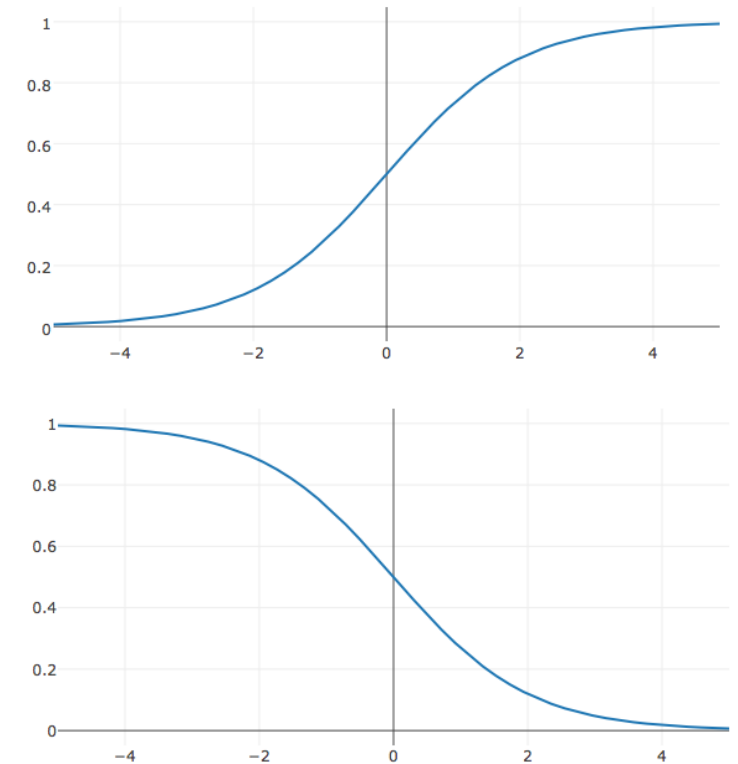


A. $w = -1$

B. $w = 1$

C. $w = -\infty$

D. $w = +\infty$



1643395



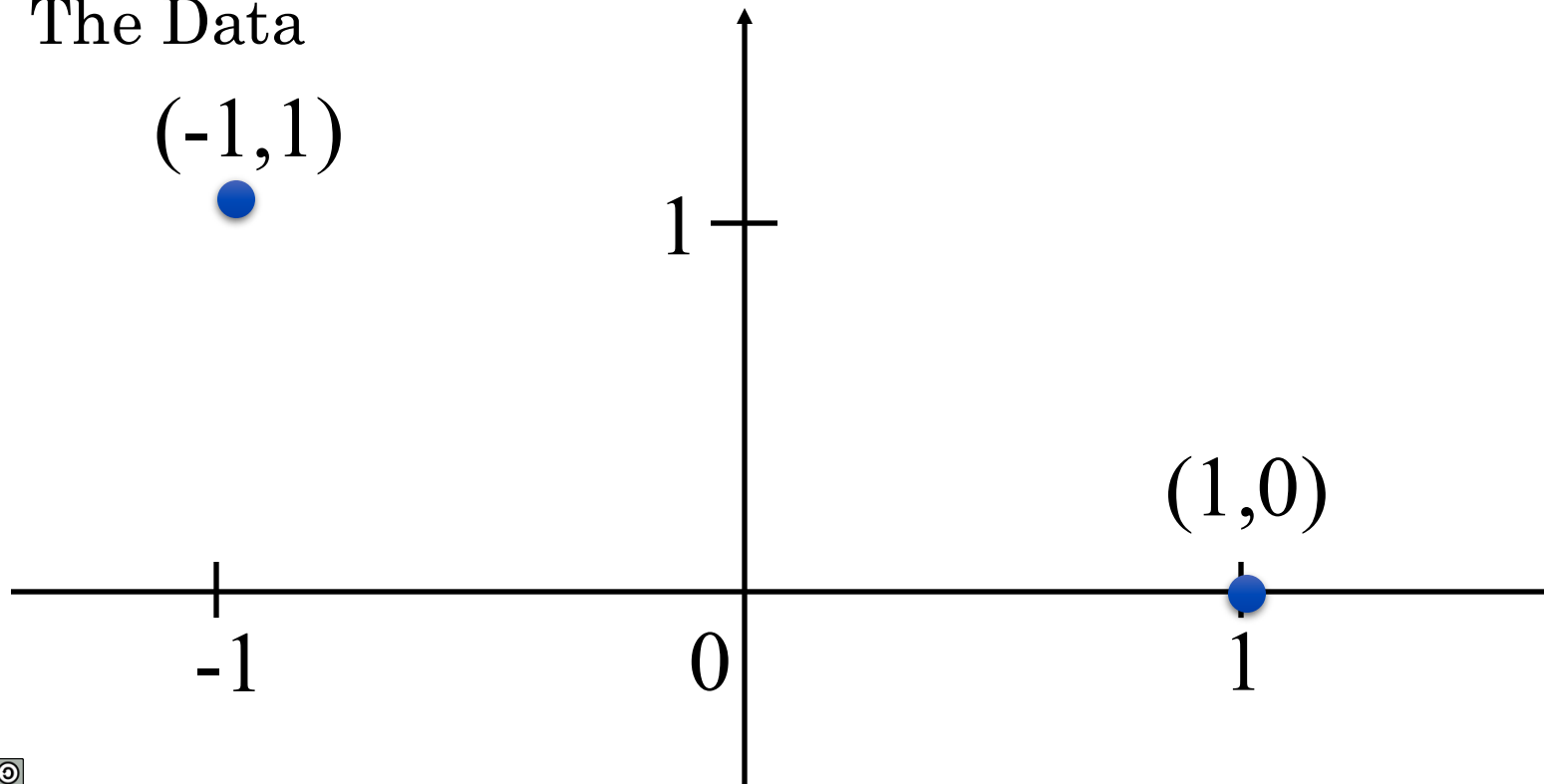
What is the best value for w ?

What Is the Value of w ?

$$w = \underset{w}{\operatorname{argmin}} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

Assume $\phi(\mathbf{x}) = \mathbf{x}$

The Data

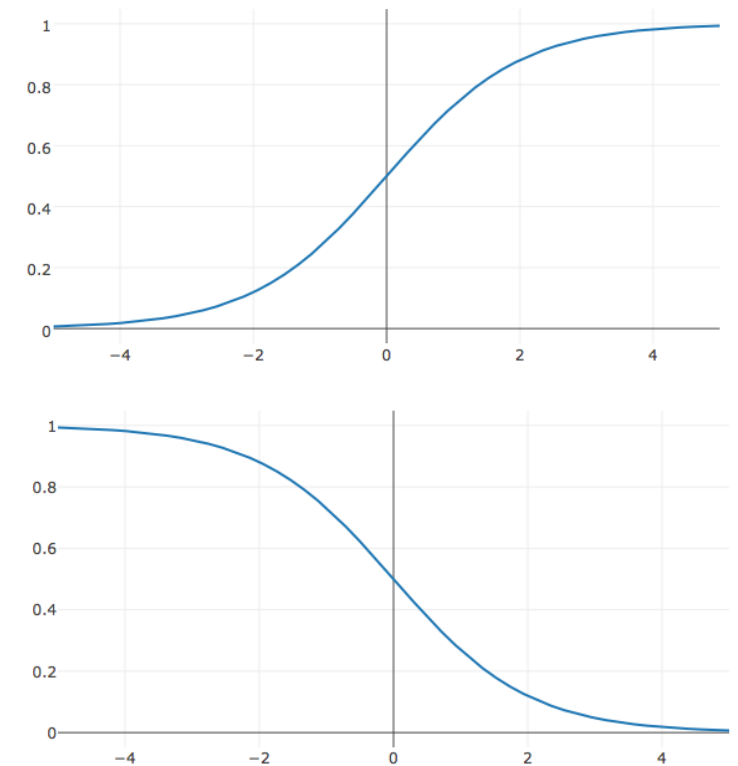


A. $w = -1$

B. $w = 1$

C. $w = -\infty$

D. $w = +\infty$



1643395

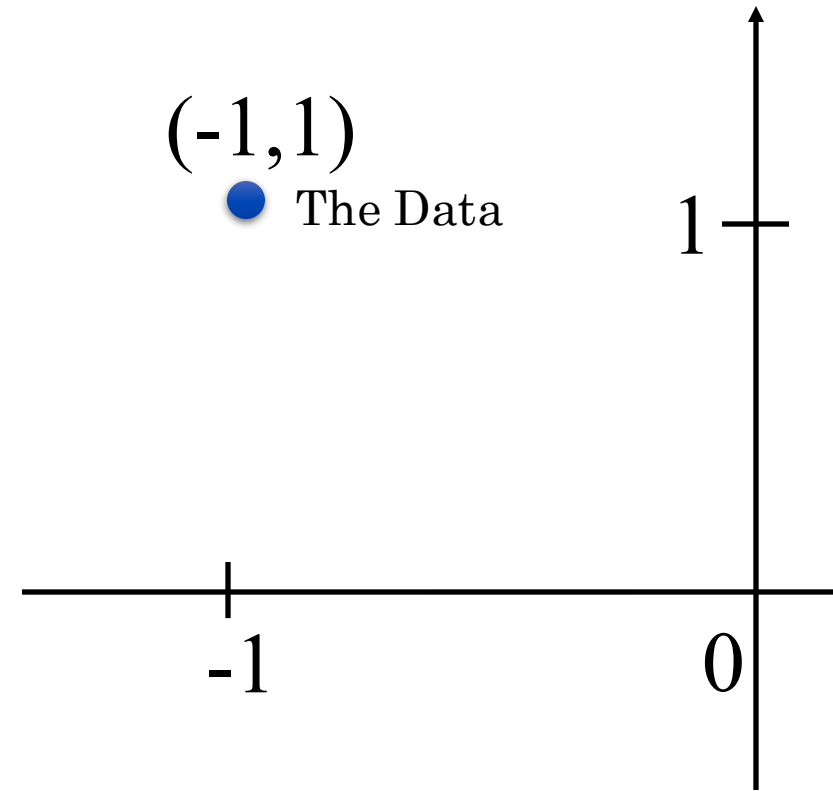
What Is the Value of w ?

$$w = \underset{w}{\operatorname{argmin}} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

For the point $(-1, 1)$:

Objective:

$$-\ln \sigma(w^T \mathbf{x}) = -\ln \sigma(-w) \Rightarrow w \rightarrow -\infty$$





1643395

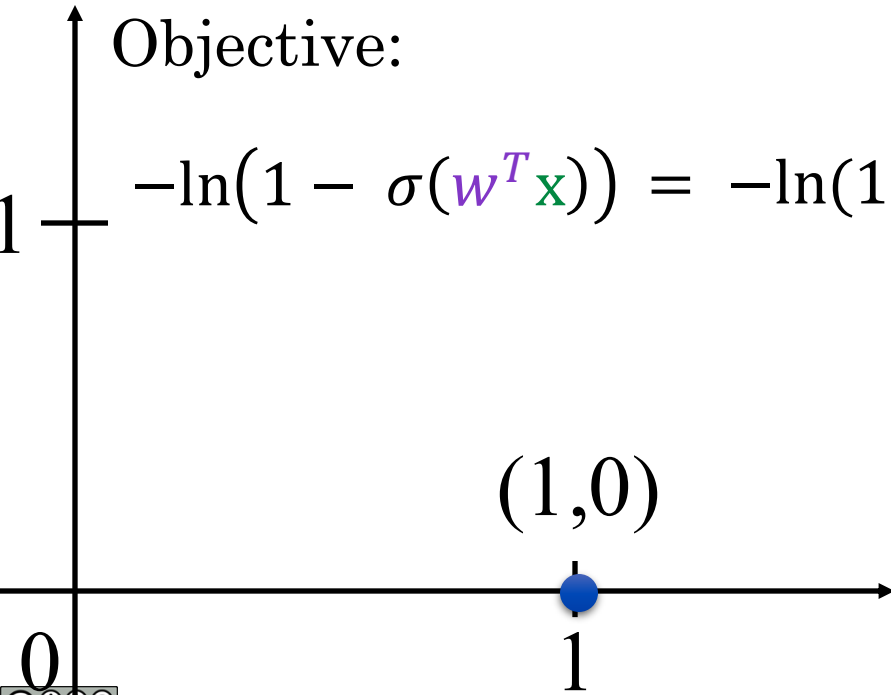
What Is the Value of w ?

$$w = \underset{w}{\operatorname{argmin}} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

For the point (1,0):

Objective:

$$-\ln(1 - \sigma(w^T \mathbf{x})) = -\ln(1 - \sigma(w)) \Rightarrow w \rightarrow -\infty$$



What Is the Value of w ?



1643395

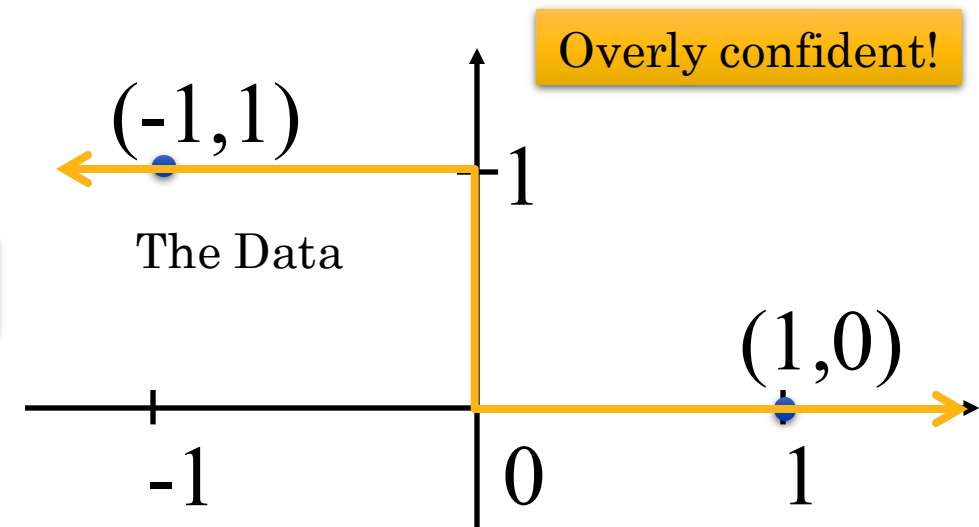
For the point $(-1,1)$:

$$-\ln \sigma(w^T \mathbf{x}) \Rightarrow w \rightarrow -\infty$$

For the point $(1,0)$:

$$-\ln(1 - \sigma(w^T \mathbf{x})) \Rightarrow w \rightarrow -\infty$$

Degenerate Solution!



Adding Regularization to Logistic Regression

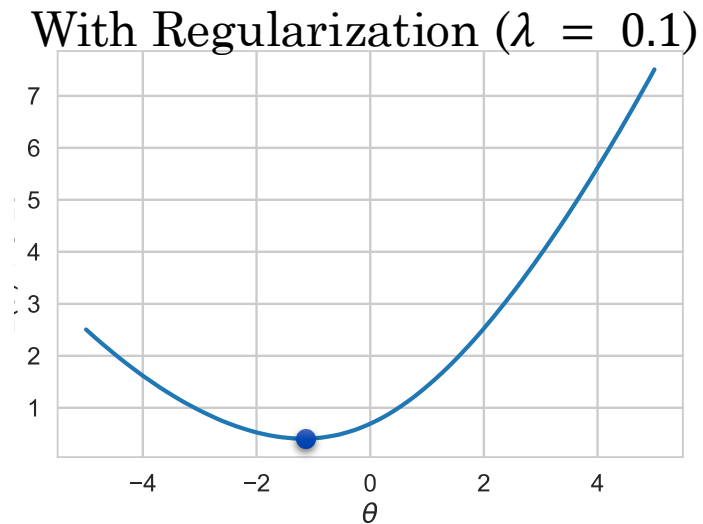
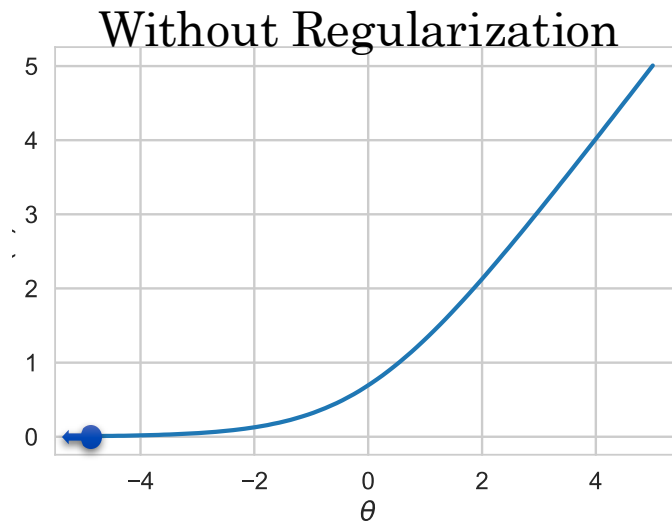


1643395

$$\underset{\mathbf{w}}{\text{argmin}} - \sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] + \frac{\lambda}{2} \sum_{d=1}^D w^2$$

Prevents weights from diverging on linearly separable data.

Earlier
Example





1643395

Logistic Regression Optimization

Logistic Regression Optimization



1643395

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = - \sum_{n=1}^N \left[\underbrace{t_n \ln y_n}_{\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}}} + \underbrace{(1 - t_n) \ln(1 - y_n)}_{\frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}}} \right]$$

Derivative wrt $\mathbf{w}$
using chain rule:

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = - \sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} + \frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}} \right]$$

Logistic Regression Optimization



1643395

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = - \sum_{n=1}^N \left[\underbrace{t_n \ln y_n}_{\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}}} + \underbrace{(1 - t_n) \ln(1 - y_n)}_{\frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}}} \right]$$

Derivative wrt $\mathbf{w}$
using chain rule:

$$\begin{aligned} \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} &= - \sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} + \frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}} \right] \\ &= - \sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} - \frac{(1 - t_n)}{1 - y_n} \frac{\partial y_n}{\partial \mathbf{w}} \right] \end{aligned}$$

$$\frac{\partial(1 - y_n)}{\partial \mathbf{w}} = - \frac{\partial y_n}{\partial \mathbf{w}}$$



1643395

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = - \sum_{n=1}^N \underbrace{[t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]}_{\substack{\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} \quad \frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}}}}$$

Derivative wrt $\mathbf{w}$
using chain rule:

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = - \sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} + \frac{(1 - t_n)}{1 - y_n} \frac{\partial(1 - y_n)}{\partial \mathbf{w}} \right]$$

$$= - \sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} - \frac{(1 - t_n)}{1 - y_n} \frac{\partial y_n}{\partial \mathbf{w}} \right]$$

$$= - \sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1 - t_n)}{1 - y_n} \right] \frac{\partial y_n}{\partial \mathbf{w}}$$

$$\frac{\partial(1 - y_n)}{\partial \mathbf{w}} = - \frac{\partial y_n}{\partial \mathbf{w}}$$



1643395

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = -\sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} + \frac{(1 - t_n)}{1 - y_n} \frac{\partial (1 - y_n)}{\partial \mathbf{w}} \right]$$

$$\frac{\partial (1 - y_n)}{\partial \mathbf{w}} = -\frac{\partial y_n}{\partial \mathbf{w}}$$

$$= -\sum_{n=1}^N \left[\frac{t_n}{y_n} \frac{\partial y_n}{\partial \mathbf{w}} - \frac{(1 - t_n)}{1 - y_n} \frac{\partial y_n}{\partial \mathbf{w}} \right]$$

$$= -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1 - t_n)}{1 - y_n} \right] \frac{\partial y_n}{\partial \mathbf{w}}$$

$y_n = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))$
Need to apply chain rule

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1 - t_n)}{1 - y_n} \right] y_n(1 - y_n) \phi(\mathbf{x}_n)$$

$$\frac{\partial y_n}{\partial \mathbf{w}} = \frac{\partial \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))}{\partial \mathbf{w}} = \sigma(\mathbf{w}^T \phi(\mathbf{x}_n)) (1 - \sigma(\mathbf{w}^T \phi(\mathbf{x}_n))) \phi(\mathbf{x}_n) = y_n(1 - y_n) \phi(\mathbf{x}_n)$$

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = -\sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$



1643395

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1 - t_n)}{1 - y_n} \right] y_n(1 - y_n) \phi(\mathbf{x}_n) \quad \text{Separating the terms}$$

$$= -\sum_{n=1}^N [t_n(1 - y_n)\phi(\mathbf{x}_n) - (1 - t_n)y_n\phi(\mathbf{x}_n)]$$

$$= -\sum_{n=1}^N [t_n\phi(\mathbf{x}_n) - \cancel{t_n y_n \phi(\mathbf{x}_n)} - y_n\phi(\mathbf{x}_n) + \cancel{t_n y_n \phi(\mathbf{x}_n)}]$$

$$= -\sum_{n=1}^N [t_n\phi(\mathbf{x}_n) - y_n\phi(\mathbf{x}_n)] = \sum_{n=1}^N [y_n - t_n]\phi(\mathbf{x}_n)$$



1643395

We Cannot Solve Directly

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}, \mathcal{D}) = -\sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)]$$

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \sum_{n=1}^N [y_n - t_n] \phi(\mathbf{x}_n)$$

$$= \sum_{n=1}^N [\sigma(\mathbf{w}^T \phi(\mathbf{x}_n)) - t_n] \phi(\mathbf{x}_n)$$

No Closed Form
Solution

$\mathbf{w}$ is inside the σ for every
term so we cannot pull out $\mathbf{w}$.

We cannot apply logit to inverse σ :
 $\text{logit}(a + b) \neq \text{logit}(a) + \text{logit}(b)$

More on this in lectures 12 and 13 with Gradient Descent.



1643395

Making Decisions



1643395

Decisions = Posteriors + Loss

- We separate *inference* from *decision*.

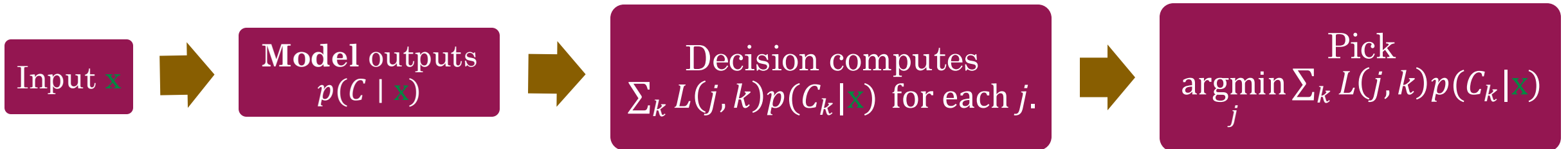
Inference gives posteriors $p(C_k|\mathbf{x})$.

The task gives a **loss matrix** L_{kj} .

L_{kj} : Loss if we predict j
when the true class is k .

The right action is the one with **minimum expected loss at this $\mathbf{x}$** .

$$\begin{array}{l} \text{Expected loss at } \mathbf{x} \text{ if} \\ \text{we choose class } j \end{array} = \sum_k L_{kj} p(C_k|\mathbf{x})$$



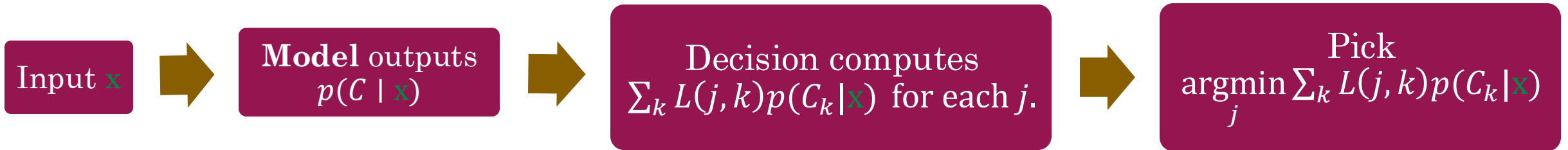


1643395

Decisions = Posteriors + Loss

The right action is the one with **minimum expected loss** at this **x**.

Expected loss at **x** if
we choose class j $= \sum_k L_{kj} p(C_k | \mathbf{x})$



Change the **costs** $L \rightarrow$ decisions change immediately;
No retraining. That's why we prefer probabilities over hard labels.



1643395

Expected Loss Example

In the example of cancer prediction, let's assume for a data sample we have:

$$p(C_0|\mathbf{x}) = 0.7 \text{ and } p(C_1|\mathbf{x}) = 0.3.$$

Assume also that the Loss matrix is given as:

	C_0	C_1
normal	0	1
cancer	100	0

If we predict normal
 $j = C_1$

$$\sum_{k=C_0} L(j, k) p(C_1|\mathbf{x}) = 100 \times 0.3 = 30$$

If we predict cancer
 $j = C_0$

$$\sum_{k=C_1} L(j, k) p(C_0|\mathbf{x}) = 1 \times 0.7 = 0.7$$

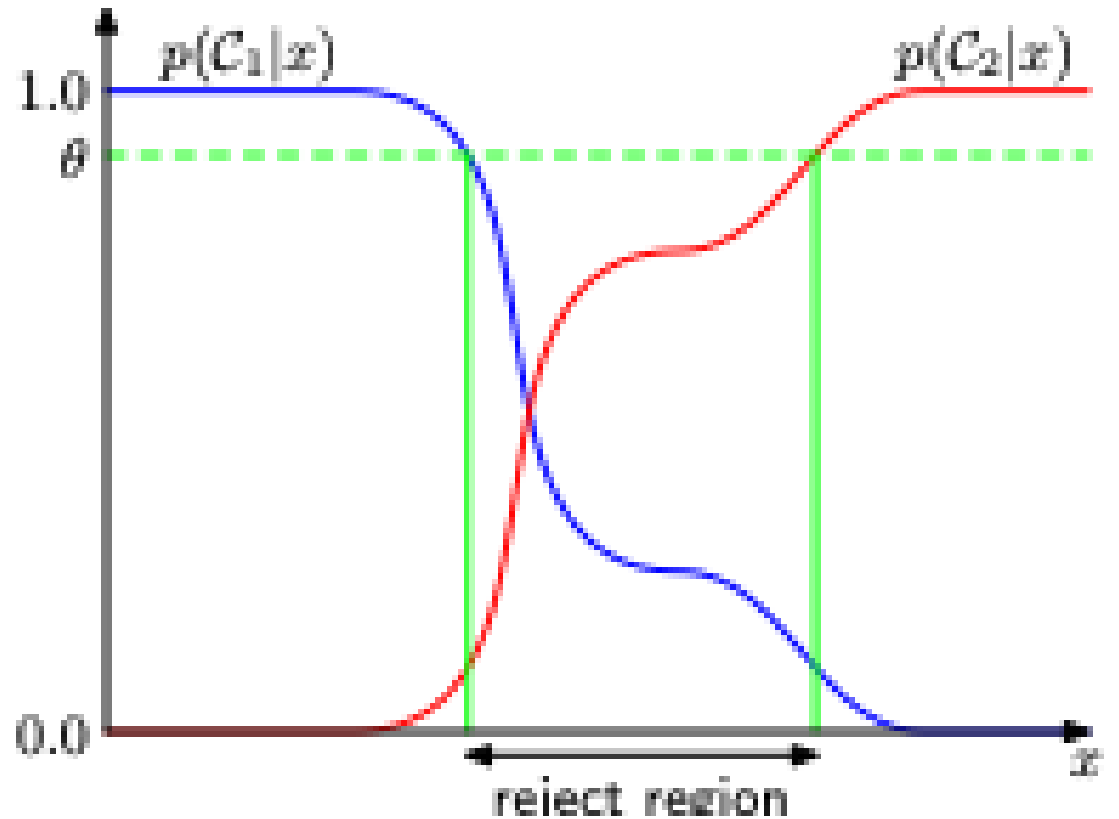
$0.7 < 30$ and predicting cancer gives smaller loss so we predict the class of cancer.

The Rejection Option



1063795

- Reject decisions when max posterior probability $\leq$ threshold θ .
- Control error rate vs. rejection fraction.





1643395

Multi-Class Classification

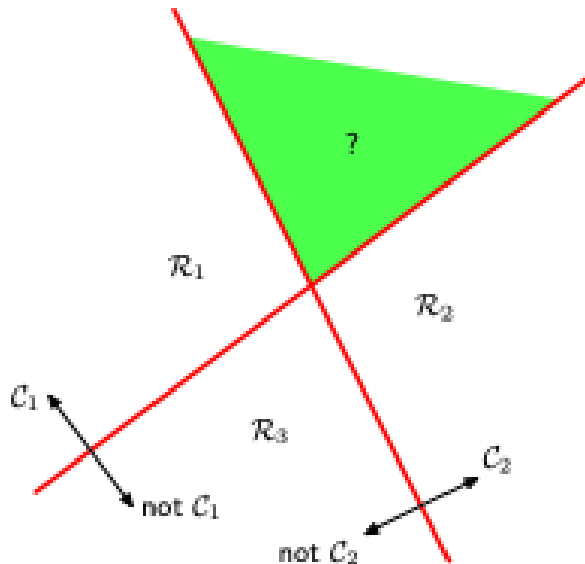


1643395

Multiple Classes

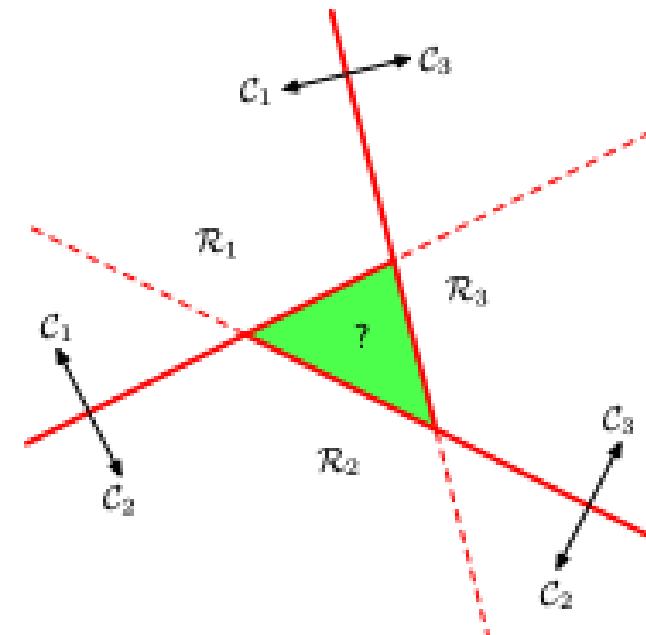
One-versus-the-rest classifier

- A model with $K - 1$ classifiers, each of which solves a two-class problem of separating points in a particular class C_k from points not in that class.



One-versus-one classifier

- Introduce $\frac{K(K-1)}{2}$ binary discriminant functions, one for every possible pair of classes.



Better Option: Multi-class Discriminant Solution



1063795

- Single K -class discriminant with linear functions:

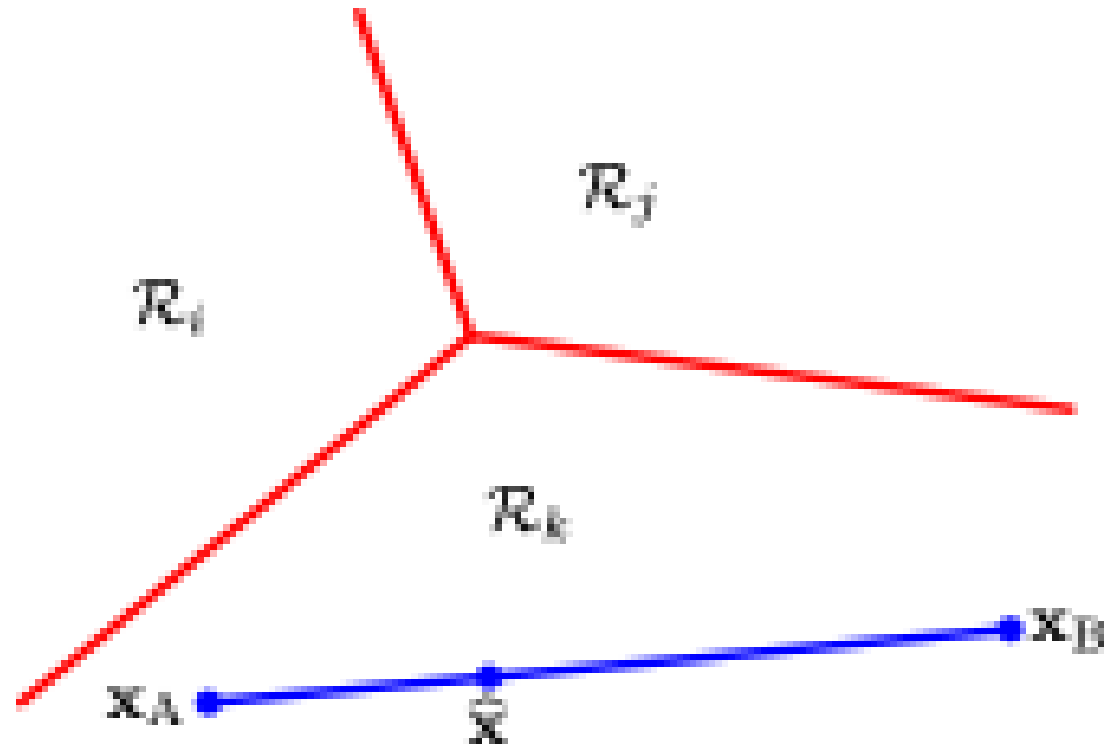
$$y_k(\mathbf{x}) = \mathbf{w}_k^T \mathbf{x} + w_{k0}$$

- Classify $\mathbf{x}$ into class C_k if:

$$y_k(\mathbf{x}) > y_j(\mathbf{x}) \quad \forall j \neq k$$

- The decision boundary between the two classes are always singly connected and convex, defined as:

$$(\mathbf{w}_k - \mathbf{w}_j)^T \mathbf{x} + (w_{k0} - w_{j0}) = 0.$$





1643395

K > 2 Classes: Softmax Function

- Let the model output **scores** (logits) $a_1, \dots, a_K \in R$.
- We want to turn these scores into $p(C_k|\mathbf{x})$.
- We generalize binary logit:

For every two class i, j : $\ln \frac{p_i}{p_j} = a_i - a_j$

- Now we solve to find p'_k s, knowing that $\sum_{k=1}^K p_k = 1$.

$$1 = \sum_{i=1}^K p_i = p_j \sum_{k=1}^K e^{a_i - a_j} \longrightarrow p_j = \frac{1}{\sum_{k=1}^K e^{a_i - a_j}} = \frac{e^{a_j}}{\sum_{k=1}^K e^{a_i}}$$

Softmax Function

$$p_j = \frac{e^{a_j}}{\sum_{k=1}^K e^{a_i}}$$

Writing the equation
based on one class j

How? $\ln \frac{p_i}{p_j} = a_i - a_j \longrightarrow p_i = e^{a_i - a_j} p_j$



Which of the following are true about the softmax function



1643395

Model Evaluation

Classifier Performance



1643395

“**Positive**” means a **prediction** of **1**.
“**Negative**” means a **prediction** of **0**.

“**True**” means **correct** prediction.
“**False**” means **incorrect** prediction.

		Prediction	
		0	1
Actual	0	True Negative (TN)	False Positive (FP)
	1	False Negative (FN)	True Positive (TP)

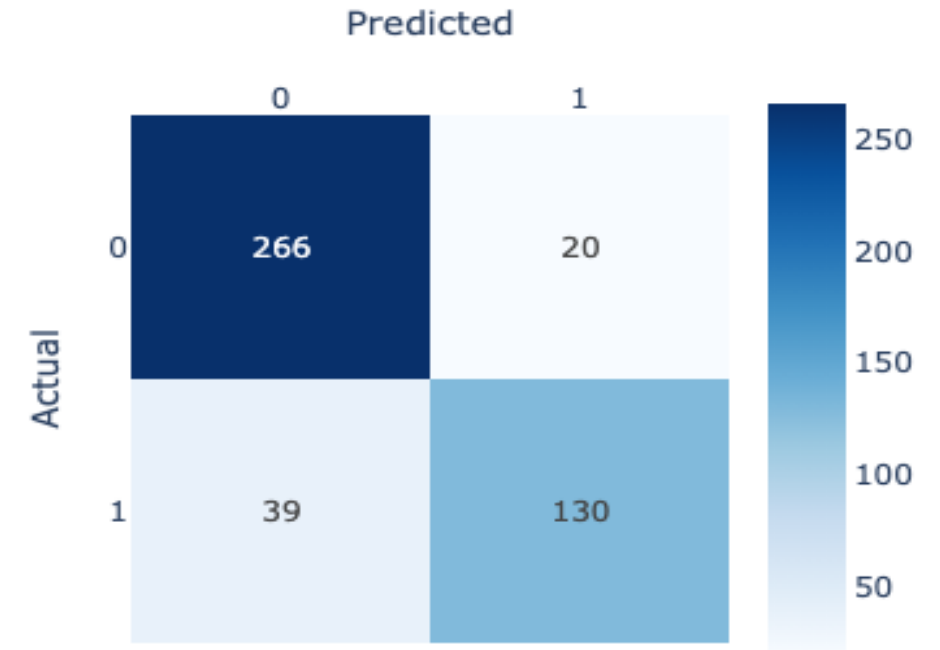
Classifier Performance



1643395

A **confusion matrix** plots TP, TN, FP, FN for a particular classifier, dataset, and **threshold (!)**.

Our **0/1 predictions** depend on our choice of probability threshold, so the confusion matrix can also **change with a new threshold!**



Classifier Accuracy



1643395

$$\text{accuracy} = \frac{\text{\# of points classified correctly}}{\text{\# Total points}}$$

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{TP + TN}{N}$$

The most basic evaluation metric for a classifier is **accuracy**.

Pitfalls of Accuracy: A Case Study



1643395

Suppose we're trying to build a classifier to filter spam emails.

Each email is **spam** (1) or **ham** (0).

We have 100 emails: **5** of them are **spam**, and the remaining **95** are real (i.e., **ham**).

Classifier 1:

Classify every email as **ham** (0).

What is the accuracy of Classifier 1?

Pitfalls of Accuracy: A Case Study



1643395

Suppose we're trying to build a classifier to filter spam emails.

Each email is **spam** (1) or **ham** (0).

We have 100 emails: **5** of them are **spam**, and the remaining **95** are real (i.e., **ham**).

Classifier 1:

Classify every email as **ham** (0).

$$accuracy_1 = \frac{95}{100} = 0.95$$

High accuracy...

...but we detected **none** ⚠ of the spam!!!

Classifier 2:

Classify every email as **spam** (1).

$$accuracy_2 = \frac{5}{100} = 0.05$$

Low ⚠ accuracy...

...but we detected **all** of the spam!!!

While widely used, the accuracy metric is **problematic** if there is **class imbalance**.



Suppose I increase the probability threshold for converting my estimated probabilities from logistic regression into 0/1 predictions.

I do not change my fitted model or test data.

What can happen to the number of true positives (TP) in the test data? Select all that apply.

Effect of changing the probability threshold on TP



1643395

I **increase** the probability threshold. What happens to the count of **true positives (TP)**?

Estimated probabilities	Threshold=0.6			
	Threshold=0.3		Threshold=0.8	
	0.21	0.63	0.85	
	↓	↓	↓	
Prediction (T=0.3)	0 (TN)	1 (TP)	1 (TP)	TP = 2
Prediction (T=0.6)	0 (TN)	1 (TP)	1 (TP)	No change!
Prediction (T=0.8)	0 (TN)	0 (FN)	1 (TP)	TP = 1

If we **increase** the threshold, it becomes **harder** to make a positive prediction.

The number of **positives (P)** will **stay the same or decrease**.

So, TP and FP will either go down or stay the same, too.

Accuracy, Precision, and Recall



1643395

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$accuracy = \frac{TP + TN}{N}$$

What proportion of **all points** were correctly classified?

"What % of emails did I classify correctly?"

$$precision = \frac{TP}{P} = \frac{TP}{TP + FP}$$

Of all **positives (P)**, what proportion were correct (**TP**)?

"What % of spam predictions were correct?"

$$recall = \frac{TP}{Actual\ 1s} = \frac{TP}{TP + FN}$$

Of all **actual 1s**, what proportion did our model **detect**?

"What % of spam emails were detected?"

Recall is also known as sensitivity.

These metrics are **threshold-dependent**.

Back to the Spam

	0	1
0	TN: 0	FP: 95
1	FN: 0	TP: 5

$$\begin{aligned} \text{accuracy} &= \frac{TP + TN}{N} \\ \text{precision} &= \frac{TP}{TP + FP} \\ \text{recall} &= \frac{TP}{TP + FN} \end{aligned}$$



1643395

Suppose we're trying to build a classifier to filter spam emails.

Each email is **spam** (1) or **ham** (0).

Let's say we have 100 emails, of which only **5** are truly **spam**, and the remaining **95** are **ham**.

Classifier 1:

Classify every email as **ham** (0).

$$\text{accuracy}_1 = \frac{95}{100} = 0.95$$

$$\text{precision}_1 = \frac{0}{0 + 0} = \text{undefined}$$

$$\text{recall}_1 = \frac{0}{0 + 5} = 0$$

Classifier 2:

Classify every email as **spam** (1).

$$\text{accuracy}_2 = \frac{5}{100} = 0.05$$

$$\text{precision}_2 = \frac{5}{5 + 95} = 0.05 \quad \left. \vphantom{\text{precision}_2} \right\} \text{Many false positives!}$$

$$\text{recall}_2 = \frac{5}{5 + 0} = 1 \quad \left. \vphantom{\text{recall}_2} \right\} \text{No false negatives!}$$

Precision vs. Recall



1643395

$$precision = \frac{TP}{TP + FP}$$

Precision penalizes FPs.

$$recall = \frac{TP}{TP + FN}$$

Recall penalizes FNs.

There is a **tradeoff** between precision and recall; they are often **inversely related**.

FPs and FNs can have **different costs** based on the Loss matrix.

- You should **adjust** your threshold to optimize for **the expected Loss**.

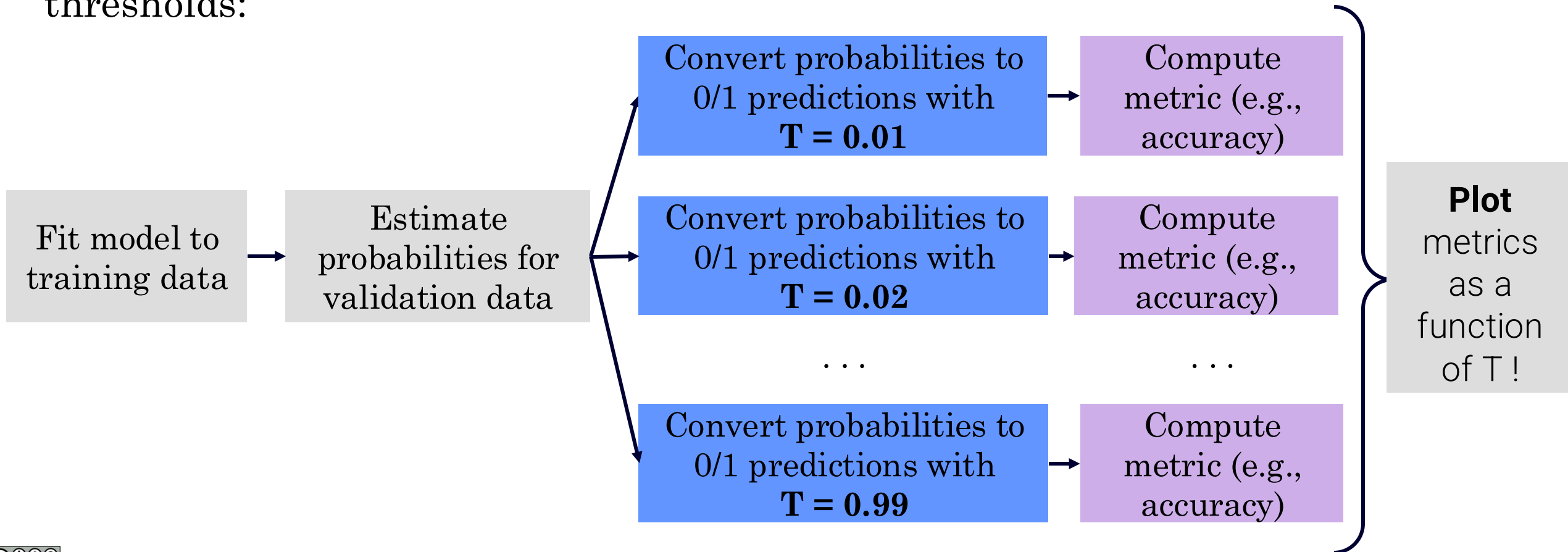
"Sweeping Through" Thresholds



1643395

In the following slides, we talk about computing metrics over different thresholds (T).

Here's a diagram of this process for a general train-validate split over 99 thresholds:



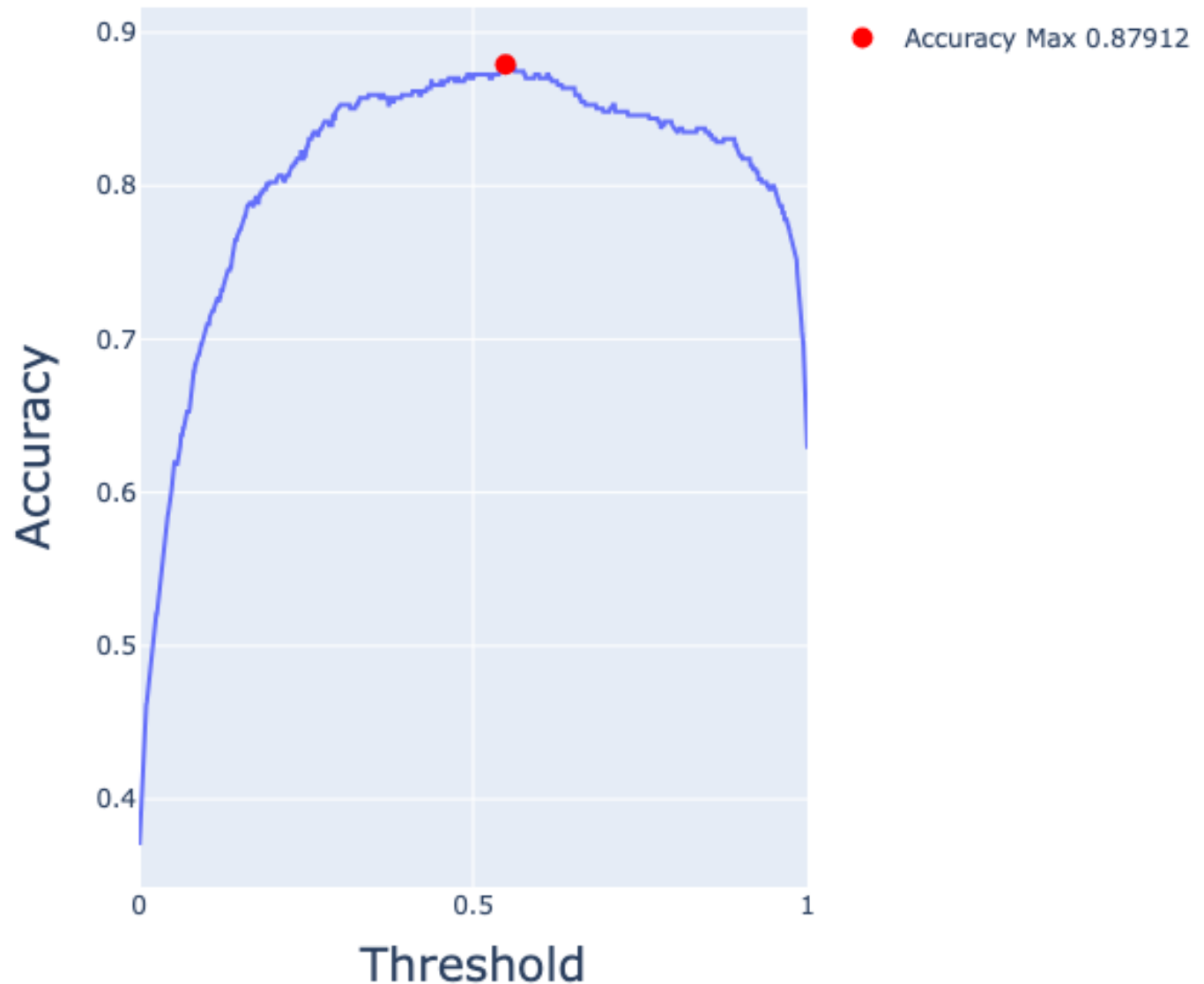
Choosing an Accuracy Threshold

The choice of threshold T impacts classification performance.

- High T : Most predictions are 0. Lots of FNs.
- Low T : Most predictions are 1. Lots of FPs.

Do we get max accuracy when $T = 0.5$? Not always the case...

Accuracy vs. Threshold



1643395

Precision-Recall Curves

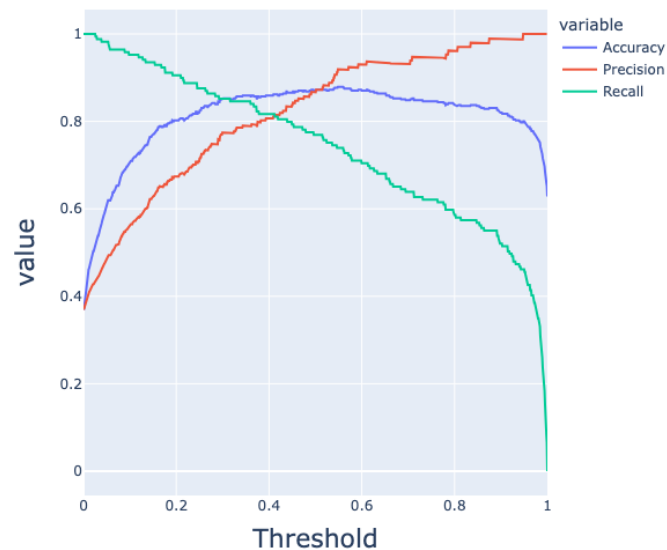


1643395

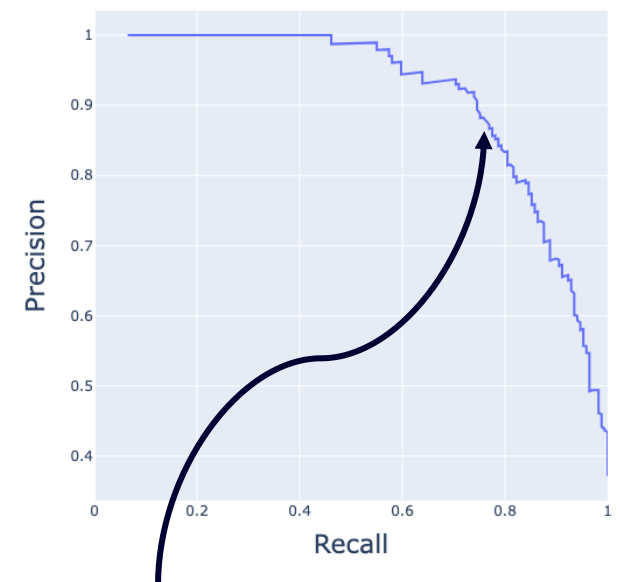
To construct a precision-recall curve, we:

- 1) Generate predictions for many different thresholds.
- 2) For each threshold, compute precision and recall.

Performance Metrics vs. Threshold



Precision vs. Recall



We often choose a threshold that keeps both precision and recall high. But, if the cost of FPs and TPs differs a lot, we may not.

Computing an F1 Score



1643395

One way to **balance** precision and recall is to maximize the F_1 **Score**:

$$\text{F1 Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- The harmonic mean of precision and recall
- Often used when there is a **large class imbalance**

How do you use it?

- Pick the threshold that **maximizes the F_1 score**

Note: If FPs and FNs have different costs, we may not want to balance precision + recall.

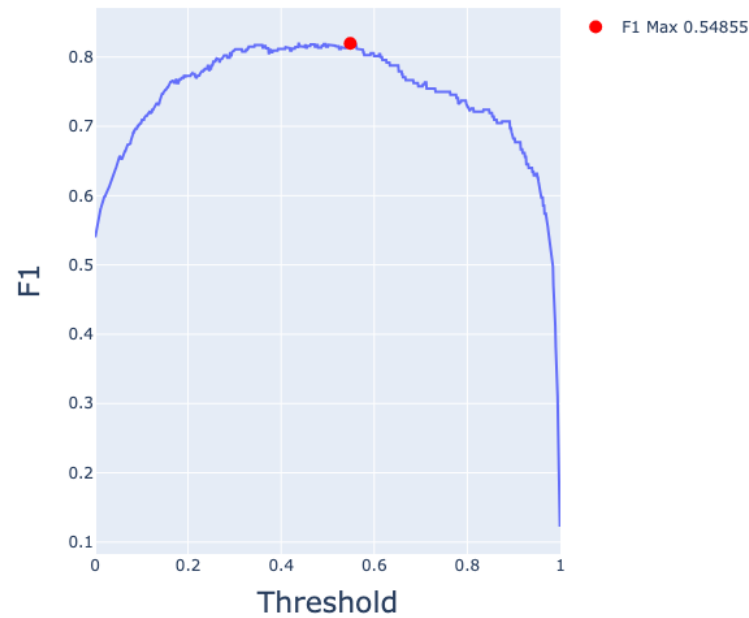
Maximizing F1 Score



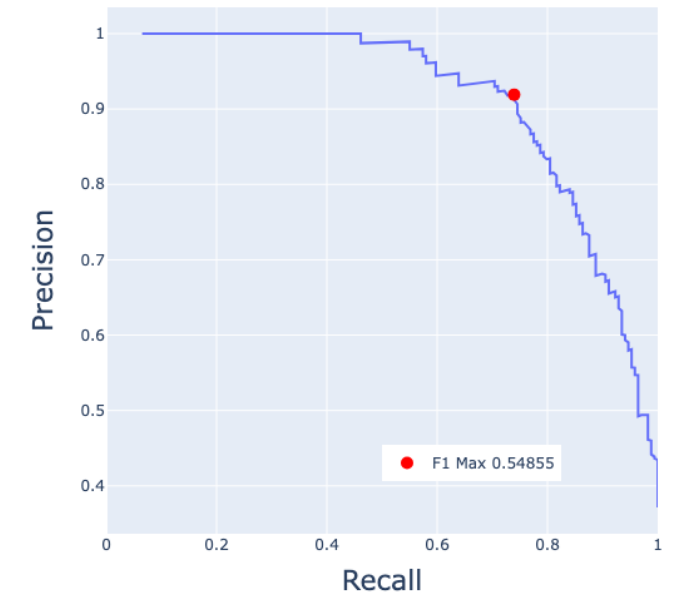
1643395

We can maximize the F1 score by evaluating the F1 score for many threshold values.

Finding F1 Score Maximum



Precision vs. Recall



True and False Positive Rates



1643395

Two more performance metrics worth knowing!

$$FPR = \frac{FP}{TN + FP}$$

False Positive Rate (FPR): Out of all actual 0s, how many did we classify incorrectly?

$$TPR = \frac{TP}{TP + FN}$$

True Positive Rate (TPR): Out of all actual 1s, how many did we classify correctly? **Same as recall.**

		Prediction	
		0	1
Actual	0	TN	FP
	1	FN	TP

Lots of classification metrics out there! All based on TP, TN, FP, FN.

ROC Curves

We can perform a similar process with FPR and TPR.

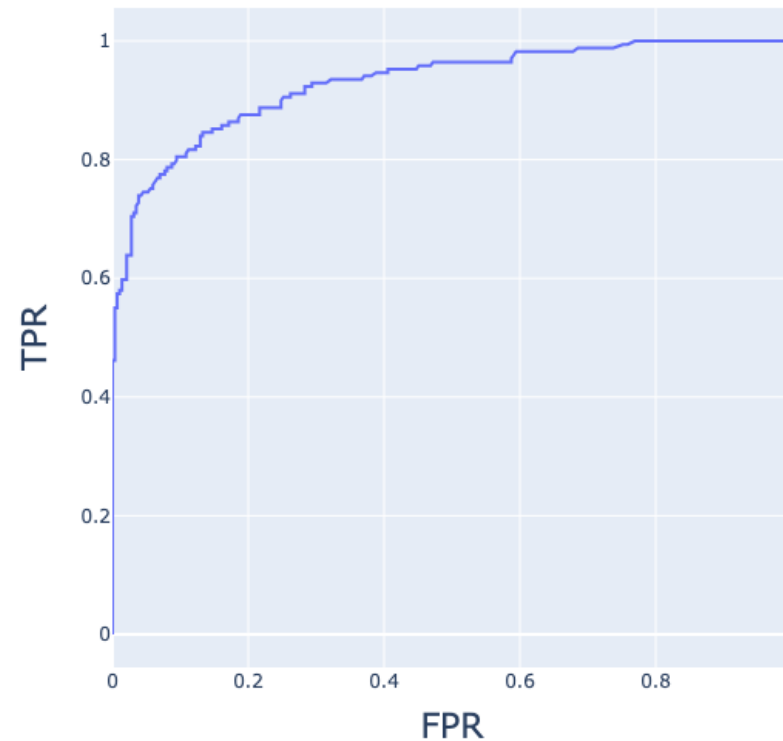
- 1) Try many thresholds
- 2) Compute the FPR and TPR for each threshold
- 3) Choose a threshold that keeps FPR *low* and TPR *high*

ROC = “receiver operating characteristic” (comes from radar in WWII)



1643395

ROC Curve



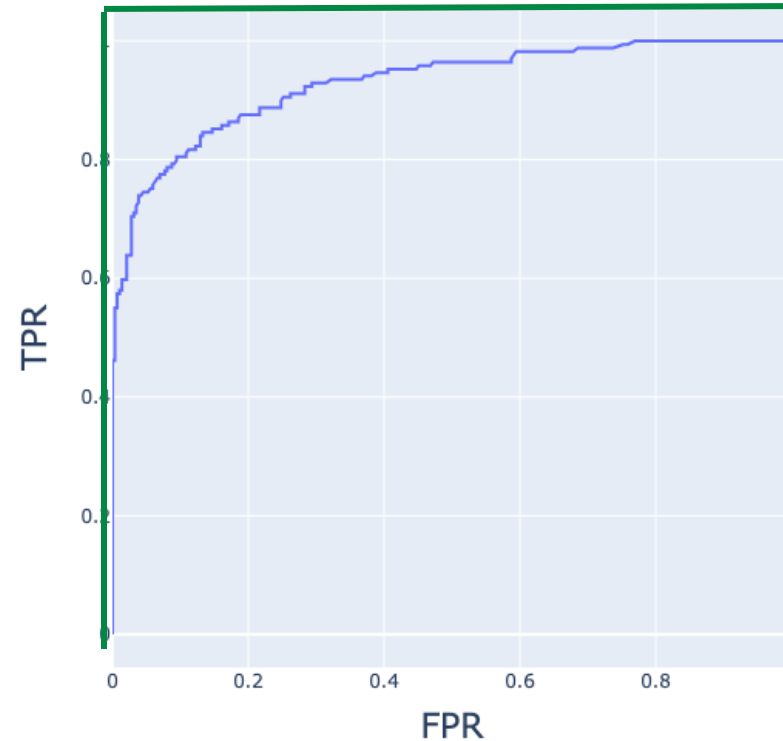
Threshold is low:
FPR high, TPR
high
(many positive
predictions)

Threshold is high:
FPR low, TPR low
(few positive
predictions)

ROC Curves

A **perfect predictor** has $\text{TPR} = 1$ and $\text{FPR} = 0$

ROC Curve



The best **Area Under the ROC Curve (AUC-ROC)** is 1.

Because we want our classifier to be as close as possible to the perfect predictor, we aim to maximize the AUC.

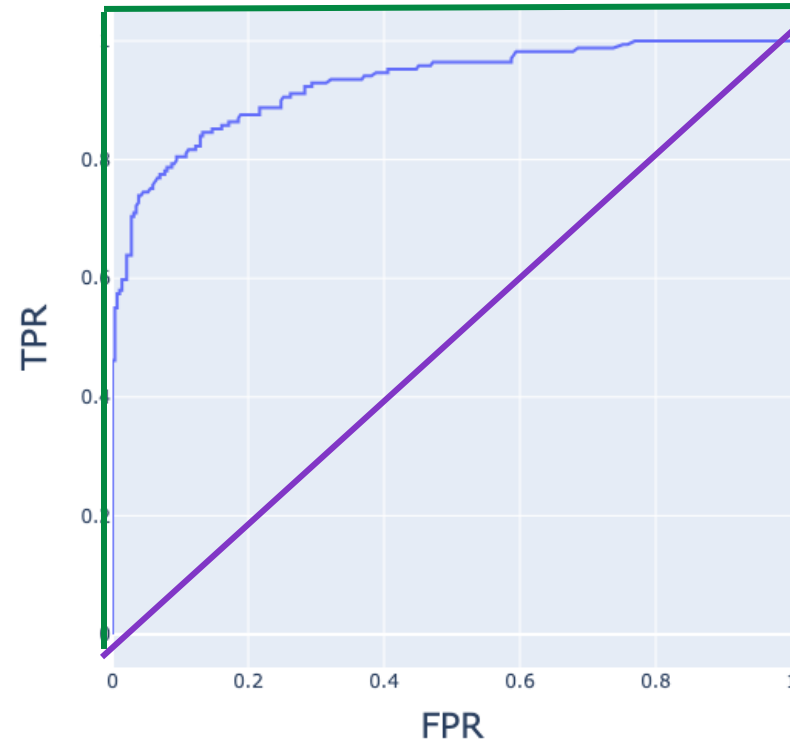


1643395

ROC Curves

A predictor that **predicts randomly** has an AUC-ROC of 0.5

ROC Curve



1643395

Real-world classifiers have an AUC between 0.5 and 1.

Summary: How do I pick my threshold?



1643395

It comes down to the cost of FPs and FNs. **Choose threshold that minimizes expected loss.**

General guidelines when FP and FN costs are similar:

When classes are **balanced** and you care equally about good negative+positive predictions:

- Pick threshold to maximize accuracy (often close to 0.5)

When classes are **imbalanced** or you care more about good **positive** predictions:

- Pick threshold to maximize F_1 score

Accuracy and **F_1 -score** are **threshold-dependent** while AUC metrics are not.

Lecture 11

Logistic Regression (2)

Credit: Joseph E. Gonzalez and Narges Norouzi

Reference Book Chapters: Chapter 5 Sections 5.1.2, 5.1.3, 5.2